

CM 8B - LA PROGRAMMATION ORIENTÉE OBJET

PROGRAMMATION PYTHON – MASTER 1 MAS

Romain Tavenard

2026

✉ romain.tavenard@univ-rennes2.fr



HÉRITAGE ET CONSÉQUENCES

1

Classe parente :

- concept générique ;
- exemple : **Polygone**

Classe fille :

- spécialisation du concept général ;
- exemple : **Rectangle**, **Triangle**, ...

Intérêt :

- la classe parente porte des attributs / méthodes qui sont communs à plusieurs classes filles ;
- les classes filles auront des spécificités

- “ Implémentez les classes *Polygone*, *Rectangle* et *Carre* ayant :
- un attribut *cotes* qui est une liste des longueurs des côtés du polygone ;
 - une méthode *perimetre* ;
 - une méthode *aire*.

”

```
class ClasseA:
    def __init__(self):
        super().__init__()
        self.a = 0

class ClasseB:
    def __init__(self):
        super().__init__()
        self.b = 0

class ClasseC(ClasseA, ClasseB):
    def __init__(self):
        super().__init__()

obj_c = ClasseC()
print(obj_c.a, obj_c.b)
```

Attention, pour que cela fonctionne correctement, il faut que toutes les méthodes de la hiérarchie fassent bien appel au constructeur de leur(s) classe(s) parente(s).

Une classe abstraite :

- doit hériter de la classe **ABC** du module **abc** ;
- ne peut pas être instanciée ;
- peut être héritée ;

C'est un **concept abstrait**.

Une classe abstraite :

- doit hériter de la classe **ABC** du module **abc** ;
- ne peut pas être instanciée ;
- peut être héritée ;

C'est un **concept abstrait**.

De la même façon, il existe :

- des méthodes abstraites (décorateur `@abstractmethod`, défini dans le module **abc**) ;
- des propriétés calculées abstraites (décorateur `@abstractproperty`, défini dans le module **abc**).

“ Implémentez une classe abstraite *Polygone* et les classes *Rectangle* et *Carre* ayant :

- un attribut *cotes* qui est une liste des longueurs des côtés du polygone ;
- une méthode *perimetre* ;
- une méthode *aire*.

”